BizQuery Final Project Report

By: Danny Guller, Viswanath Vadlamani,

Brandon Lee, and Ben Hug

All of the code is in the /doc folder as Stage_4_Check_2

Please list out changes in the directions of your project if the final project is different from your original proposal (based on your stage 1 proposal submission).

The project stayed on the same path as the Stage 1 Proposal suggested. We designed the project with the user's financial needs in mind. Some changes that we made from the original project proposal include the actual layout of the dashboard, data used by the project, and small changes to the schema for the advanced database programs.

Discuss what you think your application achieved or failed to achieve regarding its usefulness.

In regards to the usefulness of the application, the project is designed for the exact use that was previously laid out by our Project Proposal. The project allows the user to effectively track their spending. They are able to add, delete, and view their transactions. They are able to generate budget limits and receive budget alerts based on their set budget limits. They are able to filter and search transactions by date, category, amount, or payment method. We also created an interactive chart for users to know their monthly spending by category. These were laid out explicitly in the Project Proposal, and we were successful in designing this in our tool. The only area in

usefulness where the application may fail is generating financial reports every month. We had originally laid out a plan to generate a financial report, but we decided as a group that this was unnecessary.

Discuss if you changed the schema or source of the data for your application

During the initial stages of developing our tool, we realized that our real-world data was not able to fit into the schema created by our tool. As a result, we pivoted to using mostly generated data and some small amounts of real-world data. This ended up benefitting our overall project.

In order to accurately implement our transactions, triggers, and procedures, we needed to adjust our schema. We added an ArchivedTransaction table and a BudgetAlerts table. The ArchivedTransaction table is used by a stored procedure to archive older transactions. The BudgetAlerts table is used by a trigger and a transaction to keep track of when a user goes over budget and to determine how much the user goes over budget.

Discuss what you change to your ER diagram and/or your table implementations. What are some differences between the original design and the final design? Why? What do you think is a more suitable design?

The ER diagram and table implementation from Stage 2 was not changed when implemented in SQL for the application. The only difference now is the addition of the two tables mentioned above. The original design was extremely sound and was a suitable design for the application we were building. The reason we added the two new

tables is because of the Advanced SQL features, which were not introduced until Stage 4.

Discuss what functionalities you added or removed. Why?

The main functionality that we removed from the application was the ability to generate a monthly report from the application. We decided as a group that with the addition of budget alerts and a budget summary that we did not need to add the ability for a monthly report. The budget alerts and budget summaries were added to allow the user for a more “real-time” look at their current finances.

Explain how you think your advanced database programs complement your application.

The advanced database programs complement our application fairly well. The first transaction adds transactions to the application while including a budget check in the calculation. It checks the budget status for every transaction that is inserted into the database and adjusts the Budget Summary Table to reflect the real-time amount above or below the budget. Our second transaction allows the user to transfer a transaction category to another category. If the user does not want to manually update the transaction, they have the option to use the TransactionID to change the transaction category using the Transaction.

The first stored procedure allows the user to view a budget summary. This is in the form of a table inside of the “Reports” tab. The second stored procedure allows the user to archive old transactions. The user can choose how many months they would like

to archive, and those transactions will be added to the ArchivedTransactions table. This makes for a cleaner view of the “Transactions” tab.

The first trigger prevents all budgets from being negative. If a user tries to add a negative budget, then the database rejects the budget. You will not find the budget added to the “Budgets” tab. The second trigger checks the budget after a transaction is inserted. If it is inserted, the trigger will send an alert that is caught by our server in the code. The server will then send a budget alert to the user, which can be viewed inside of the “Budget Alerts” table.

The constraints are the exact primary and foreign keys as listed in Stage 2. They are implemented and used in the exact way we laid out. All of these advanced database programs complement each other and work very well with the application.

Each team member should describe one technical challenge that the team encountered. This should be sufficiently detailed such that another future team could use this as helpful advice if they were to start a similar project or where to maintain your project.

Ben - One of the hardest technical challenges we faced was learning and using HTML in a way that makes the application easy to use. We found different templates and documentation online to help us form a small framework for what the application would look like. That documentation was extremely helpful and definitely allowed us to design an application with a modern look.

Viswanath - Main technical challenge was designing the ER diagram and ER diagram in a way that would hold as new features were added. Early on, I realized that even small

mistakes in entity relationships can create issues later in the backend and UI. The biggest lesson I learnt is to thoroughly validate schema before writing any SQL, making sure every entity, attribute, and key is truly needed and supports actual user flows.

Danny - There was a challenge with getting alerts from the trigger that was implemented inside of the Database, and to resolve it, learning how to grab that alert and make a useful part of our application unlocked a big part of the tool. Our ability to learn how this advanced database functionality worked gave us the ability to make the application better and more user-friendly.

Brandon - One of my hardest challenge this project was getting the backend to work with the pie chart lesson. I was able to overcome this challenge with lot of googling and research. The best way I worked effectively in overcoming this challenge was starting early, as this allowed me the time to debug this problem, as we believe visualizations is one of the most important aspect of our solution.

Are there other things that changed comparing the final application with the original proposal?

The only other thing that changed from the final application with the original proposal was the emphasis on the user's income. We did not implement a way to calculate or report the user's income for analysis. We mainly focused on the user's current spending and budgets.

Describe future work that you think, other than the interface, that the application can improve on

The application can improve on the usability of the different budgets. The use and update of the budgets inside the application is extremely rudimentary. The future work would include adding more categories for budgets, adding a function to add weekly or yearly budgets, and the ability to incorporate machine learning to predict monthly spending based on current spending habits.

Describe the final division of labor and how well you managed teamwork.

The team worked very well together. We had regular meetings to discuss the different stages of the project and the different parts of the code. Everyone contributed to the actual development of each coding and design stage. In the same vein, everyone was present at all team meetings and helped the team, for both in person and virtual meetings. The original Project Proposal included each team member and their respective roles. Each member of the team respected their roles and contributed to parts of the project outside of their respective areas.

Stage 2 and Stage 3

We have not received feedback for either stage.